Finalmente chegou a hora de criarmos a nossa criptomoeda Turing, e com base nela, nós desenvolveremos uma dApp para votarmos nos projetos finais da disciplina.

Para isso, vamos tomar como base a dApp da atividade anterior (atividade 12) e fazer mudanças nela!

> ORIENTAÇÕES GERAIS DE EXECUÇÃO E TESTE: Nesta ENTREGA, vocês usarão a blockchain local criada pelo HardHat. Com isso, os usuários e addr criados serão "FAKE", conforme a tabela exemplo.

Nome do contrato/token: Turing

> **[ATENÇÃO] ao mudar o nome do contrato, vc deverá alterar o arquivo**
>
> **[bolilerplate]/scripts/deploy.js … nas Linhas 25 e 48**

Este contrato utilizará a estratégia " Uncapped Lazy Supply", isto é, os tokens não são emitidos de uma só vez durante o deploy, mas sim criados sob demanda em quantidades menores, sem qualquer limite máximo predefinido no código. Especificamente neste nosso caso, **não haverá nenhuma initial coin offer (sem minting no construtor!)**.

Controle de acesso

Nem todos os usuários poderão utilizar nosso smart contract. Apenas usuários pré-autorizados.

Desta forma, o contrato terá um **mapping**, com os codinomes mapeando em endereços de cada **usuário autorizado:**

```
mapping(string => address) public users;
```

Para esta primeira fase, estes usuários serão os da tabela exemplo. Você deve inicializar esse mapping, no construtor, com os dados da tabela.

Métodos públicos do contrato

function updateUser(codinome, user_addr)

- **Requires**
  - Este método poderá ser executado apenas pelo owner ou a professora.
    - **Atenção!!** Esse é um require que se repete em outras funções… com isso você DEVE utilizar um modfier para impelementá-lo.
  - codinome não pode ser uma string vazia.
- **O que ela faz:** Atualiza o address do user codinome dentro do mapping “users”.

function votingOn()

- **Requires**
  - Este método poderá ser executado apenas pelo owner ou a professora.
- **O que ela faz:** Seu contrato deverá ter uma variável booleana "**voting**", inicializada em true, indicando se a votação está ativa. Após sua execução, essa variável é setada para true, indicando que a votação está ativa.

function votingOff()

- **Requires**
  - Este método poderá ser executado apenas pelo owner ou a professora.
- **O que ela faz:** Após sua execução, a variável booleana citada deve ser setada para false, indicando que a votação está pausada.

function issueToken(codinome , qtde_de_Turings)

- **Requires**
  - Este método poderá ser executado apenas pelo owner ou a professora.
- **O que ela faz:** Ele criará (*minting!*) a quantidade especificada de Turings na carteira do "codinome".

function vote(codinome, qtde_de_Turings)

- **Requires**
  - A votação deve estar ativa.
  - Apenas usuários pré-autorizado podem executá-la (os da tabela exemplo).
  - Um usuário NÃO pode votar em si mesmo.
  - qtde_de_Turings deve ser maior que 0, e menor ou igual a 2**.**
  - Um mesmo usuário só pode votar **uma vez** em um mesmo codinome.
    - **DICA:** você poderá implementar um mapping de mappings (mapeamento aninhado):

```
// Chave primária (string) => (Chave Secundária (string) => Valor Final (bool))
mapping(string => mapping(string => bool)) public hasVote
```

- **O que ela faz:** Quem chamar este método, estará “votando” e gerando (*minting!*) uma quantidade de **Turings** para o codinome. Além disso, você deverá implementar uma lógica de forma que a cada 10 vezes que um usuário votar, esse usuário **receberá 1 Turing**!

VALENDO TURINGS EXTRA: implementar o Turing como um ERC-20 (isso implicará em várias pequenas mudanças do HardHat e nos arquivos e configuração js)... 2 ou 3 turings, dependendo do mercado :-)

FRONTEND

O caminho mais fácil é desenvolvê-lo com base na Atividade 12, em javascript. Mas fiquem à vontade para alterá-lo completamente, inclusive utilizando outros frameworks/linguagens (eg. python)...

Seu *frontend* deverá oferecer as seguintes funcionalidades:

- Apresentar interfaces diferentes para usuários comuns (registrados) e os “superusuários”:
  - Para usuários comuns: opção para chamar a função **vote.**
  - Superusuários: além da função vote, opção para chamar as funções **issueToken**, **votingOn** e **votingOff.**
- Tanto no caso da função vote, quanto no caso da função issueToken, deve haver um campo para **selecionar** um dos Codinomes de aluno (excluir a professora!), e outro campo para inserir a quantidade de Turings...
- **Exibir o ranking de usuários, por ordem de Turings.** Aqui existem duas possíveis implementações:
  - (i) a forma "braçal", fazendo *pooling* no balanceOf() do contrato;
  - (ii) **ou usando Eventos** (https://docs.alchemy.com/docs/solidity-events). Nesse último caso, por exemplo, sempre que houver uma chamada à função vote(), o dApp captura o evento e atualiza o ranking.

ENTREGA

Preencha o forms a seguir com o link do seu projeto no GitHub

- https://forms.gle/tisTeNnmTtSSWSML7

DÚVIDAS COM SOLIDITY

https://www.dappuniversity.com/articles/solidity-tutorial

https://docs.soliditylang.org/en/latest/